Databases & SQL for Social Data

Yasin Shafi

2 April 2026

Note on data. This week uses **synthetic** (simulated) campaign finance-style data generated in the course script. The goal is to practice database thinking and SQL—not to draw substantive inferences about real campaigns.

Start Off: Verifying Your Environment

1. **Environment check (required).** Submit proof that you successfully ran the full tutorial code on your machine. You may submit *one* of the following:
   - A screenshot or text file showing console output that includes the printed representation shapes (document–term matrix, Word2Vec document vectors, and transformer embeddings).

Conceptual Questions

Please write three to ten sentence explanations for each of the following questions. **You are only required to answer ONE of the two questions below.**

2. Explain what a **relational schema** is and why it is useful for social data. In your answer, define **primary keys** and **foreign keys**, and explain how they reduce duplication and enable joins. Use the (candidate, contributor, contribution) setting from this week's coding lab as your concrete example.

Response: A relational schema is a blueprint that defines how tables are structured and how they relate to each other through keys. A primary key is a unique identifier for each row in a table. For example, each candidate gets a candidate_id and each contributor gets a contributor_id. A foreign key is a column in one table that points to the primary key of another. The contribution table holds both candidate_id and contributor_id as foreign keys, linking all three tables together. This design eliminates duplication because candidate and contributor details are stored only once in their own tables rather than repeated across every contribution record. When we want to answer a question like "how much did each contributor give to each party," we use a SQL join to reassemble the tables at query time using those keys.

Applied Exercises

Use the code in the week's code tutorial and the lecture slides to answer the following questions.

Response: Please find this week's response here: `https://github.com/yasinshafi/soda501/tree/main/11_databases_and_sql/problem_set`.

4. **Build the database + inspect the schema (synthetic data).** Using the provided @MastersThesis•, author = •, title = •, school = •, year = •, OPTkey = •, OPTtype = •, OPTaddress = •, OPTmonth = •, OPTnote = •, OPTannote = • script, create the SQLite database (`campaign_finance.db`) and load the synthetic tables: `candidates`, `contributors`, and `contributions`.
   - Report the row counts in each table using `SELECT COUNT(*)`.
   - Show the schema for each table (e.g., `PRAGMA table_info(candidates)` and similarly for the other two tables).
   - Briefly explain (2–4 sentences) how `contributor_id` and `candidate_id` connect the tables.

     Response: How contributor_id and candidate_id connect the tables: The contributions table acts as a bridge between contributors and candidates. Each row in contributions holds a contributor_id (a foreign key referencing contributors.id) and a candidate_id (a foreign key referencing candidates.id). This means we don't need to store contributor or candidate details more than once. We can store them in their own tables and link to them through these keys. Any query that needs information from all three tables chains two joins: contributions -¿ candidates via candidate_id, and contributions -¿ contributors via contributor_id.

5. **Joins + aggregation (write and run your own SQL).** Write a SQL query (and run it through R or Python) that uses at least **one join** and at least **one aggregation**:
   - Required: join `contributions` to `candidates` and compute total contributions by `party`.
   - Required: restrict to contributions with `amount > 1000`.
   - Output: a clean table with columns `party`, `total_amount`, and `num_contributions`.
   - Visualization: make a simple bar plot of `total_amount` by party.
6. **Indexes + query plan (evidence of optimization).** Using SQL statements, do the following:
   - Verify which indexes exist on `contributions` (e.g., query `sqlite_master`).

- Choose one query that filters by `candidate_id` *or* `date` *or* `amount`. Run `EXPLAIN QUERY PLAN` for that query.
- In 4–6 sentences, interpret the query plan: does SQLite report using an index? If not, what index might help and why?

  Response: Query plan interpretation: The EXPLAIN QUERY PLAN output shows whether SQLite uses an index or falls back to a full table scan (SCAN TABLE). For the filter on amount > 1000, SQLite uses the idx_contrib_amount index on the contributions table, meaning it does not read every row, it jumps directly to rows satisfying the condition. The join to candidates on candidate_id similarly benefits from the idx_contrib_candidate_id index, allowing SQLite to look up matching candidate rows efficiently rather than scanning the entire candidates table for each contribution row. If these indexes did not exist, both operations would require full table scans, which at 1 million rows would be substantially slower.

7. **Challenge Question (Optional — if you finish early):** Run the same join-and-aggregate query in **DuckDB** (from R or Python) and compare it to SQLite.
   - Repeat the query in DuckDB (you may load the tables from CSV or connect to the SQLite database, depending on what you set up in lab).
   - Provide one piece of evidence about performance (e.g., a timing using `system.time()` in R or timing in Python).
   - In 3–6 sentences, explain what factors might drive differences (data size, indexing, execution engine, I/O, query planner).